

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY7. 딥러닝 다중 퍼셉트론

목 차

1. 딥러닝 다중 퍼셉트론 (MLP)		3
2. 딥러닝 최적화 함수		12
3. 딥러닝 RNN 모델		21
4. 딥러닝 GAN 모델		53

1. 딥러닝 다중 퍼셉트론 (MLP)

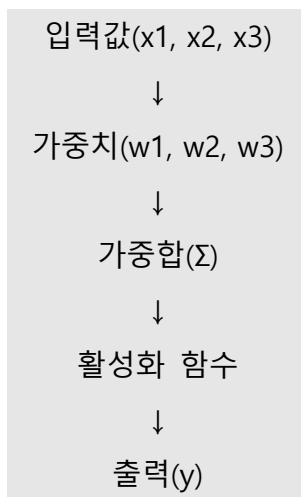
다중 퍼셉트론(MLP)은 인공신경망(Artificial Neural Network)의 가장 기본적인 형태로, 입력층과 출력층 사이에 하나 이상의 은닉층(Hidden Layer)을 두어 복잡한 문제를 학습할 수 있도록 만든 딥러닝 모델이다.

1. 퍼셉트론(Perceptron)이란?

퍼셉트론은 인간의 뉴런을 수학적으로 모방한 가장 단순한 인공신경망 모델이다.

입력값을 받아 가중치를 곱한 후 모두 더하고 활성화 함수를 적용하여 결과를 출력한다.

구조는 다음과 같다.



수식으로 표현하면

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

여기서

- x : 입력값
- w : 가중치
- b : 편향(Bias)
- z : 가중합

출력은

$$y = f(z)$$

로 계산된다.

2. 단층 퍼셉트론의 한계

단층 퍼셉트론은 선형 분리 가능한 문제만 해결할 수 있다.

예를 들어

- AND 게이트
- OR 게이트

는 학습 가능하다.

하지만

- XOR 게이트

는 학습할 수 없다.

XOR 문제

A B 출력

0 0 0

0 1 1

1 0 1

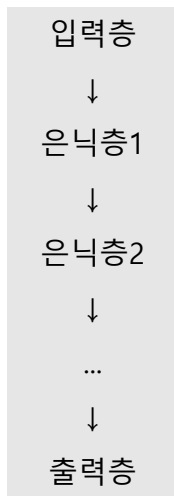
1 1 0

XOR은 하나의 직선으로 구분할 수 없기 때문에 단층 퍼셉트론으로는 해결되지 않는다.

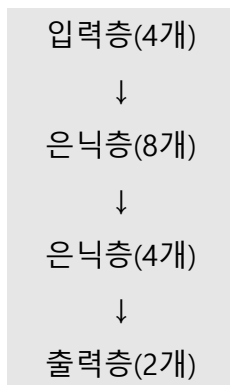
이 문제를 해결하기 위해 등장한 것이 다층 퍼셉트론이다.

3. 다층 퍼셉트론(MLP)의 구조

MLP는 입력층과 출력층 사이에 은닉층을 추가한 구조이다.



예시

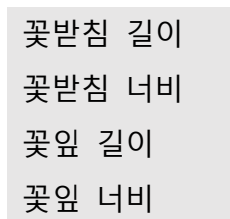


4. MLP의 계층 구조

(1) 입력층(Input Layer)

데이터를 입력받는다.

예를 들어 붓꽃(Iris) 데이터라면

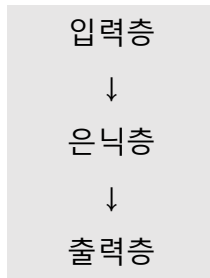


총 4개의 입력 노드가 필요하다.

x1 x2 x3 x4

(2) 은닉층(Hidden Layer)

입력 데이터의 특징을 추출하는 역할을 수행한다.



은닉층이 많을수록 복잡한 패턴을 학습할 수 있다.

(3) 출력층(Output Layer)

최종 결과를 출력한다.

예를 들어

이진 분류

고양이 / 개
출력 노드 수
1

다중 분류

setosa
versicolor
virginica
출력 노드 수
3개

5. MLP의 계산 과정

예를 들어

입력층 2개

은닉층 2개

출력층 1개

구조라고 가정한다.

1단계

입력

$x_1=1$

$x_2=2$

2단계

은닉층 계산

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2)$$

3단계

출력층 계산

$$y = f(v_1h_1 + v_2h_2 + b_3)$$

6. 활성화 함수(Activation Function)

MLP의 핵심은 활성화 함수이다.

만약 활성화 함수를 사용하지 않으면 여러 층을 쌓아도 선형 모델과 동일해진다.

Sigmoid

수식

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

특징

- 출력 범위 0~1
- 확률 표현 가능
- 기울기 소실 문제 발생

Tanh

수식

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

특징

- 출력 범위 -1~1
- Sigmoid보다 성능 우수

ReLU

수식

$$ReLU(x) = \max(0, x)$$

특징

- 계산이 단순
- 학습 속도가 빠름
- 현재 가장 많이 사용

Leaky ReLU

수식

$$f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

특징

- 죽은 ReLU 문제 해결

Softmax

수식

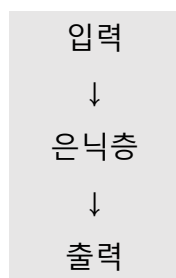
$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

특징

- 다중 분류 출력층에서 사용
- 출력값의 합이 1

7. 순전파(Forward Propagation)

입력층 → 출력층 방향으로 계산하는 과정이다.



과정

1. 가중합 계산
2. 활성화 함수 적용
3. 다음 층 전달
4. 최종 출력 생성

8. 손실 함수(Loss Function)

예측값과 실제값의 차이를 계산한다.

평균제곱오차(MSE)

회귀 문제

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

교차 엔트로피(Cross Entropy)

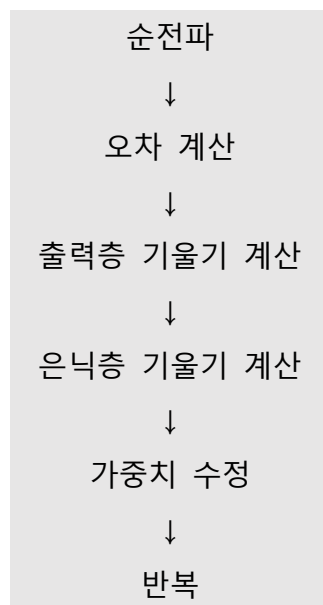
분류 문제

$$L = - \sum_i y_i \log(\hat{y}_i)$$

9. 역전파(Backpropagation)

MLP 학습의 핵심 알고리즘이다.

과정



10. 경사하강법(Gradient Descent)

가중치를 수정하여 손실함수를 최소화한다.

가중치 갱신 공식

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

여기서

- L : 손실 함수
- η : 학습률(Learning Rate)
- $\frac{\partial L}{\partial w}$: 기울기

11. MLP의 장점

1. 비선형 문제 해결 가능
2. XOR 문제 해결 가능
3. 다양한 분류·회귀 문제 적용 가능
4. 딥러닝의 기본 구조 제공
5. 복잡한 패턴 학습 가능

12. MLP의 단점

1. 데이터가 많아야 성능이 좋음
2. 학습 시간이 길 수 있음
3. 과적합 발생 가능
4. 이미지에서는 CNN보다 비효율적
5. 시계열에서는 RNN, LSTM보다 비효율적

13. 실제 활용 분야

이미지 분류

손글씨 숫자 분류

품질 검사

금융

대출 심사

신용 평가

사기 탐지

의료

질병 진단

환자 분류

제조

불량품 판정

예측 유지보수

다층 퍼셉트론(MLP)은 입력층, 하나 이상의 은닉층, 출력층으로 구성된 가장 기본적인 딥러닝 신경망이다. 입력 데이터는 순전파를 통해 예측값을 생성하고, 손실 함수를 이용하여 오차를 계산한 후 역전파와 경사하강법을 통해 가중치를 반복적으로 수정한다. 이러한 과정을 통해 복잡한 비선형 패턴을 학습할 수 있으며, 현대 딥러닝의 CNN, RNN, Transformer와 같은 고급 신경망 구조의 기초가 되는 핵심 모델이다.

2. 딥러닝 최적화 함수

딥러닝에서 최적화 함수(Optimizer) 는 신경망이 학습하면서 손실 함수(Loss Function)의 값을 최소화하도록 가중치(Weight)와 편향(Bias)을 업데이트하는 알고리즘이다.

즉,



최적화 함수는 역전파에서 계산된 기울기를 이용하여 다음 학습 방향을 결정한다.

왜 최적화 함수가 필요한가?

신경망은 처음에 가중치를 무작위(Random)로 생성한다.

예를 들어

실제값 = 100

예측값 = 20

이라면 오차가 매우 크다.

학습을 반복하면서

20 → 45 → 70 → 90 → 99

처럼 예측값이 실제값에 가까워져야 한다.

이를 위해 가중치를 조금씩 수정하는데 이 작업을 수행하는 것이 Optimizer 이다.

경사하강법(Gradient Descent)

모든 최적화 알고리즘의 기본이 되는 방법이다.

손실 함수의 기울기를 계산하여 손실이 감소하는 방향으로 이동한다.

가중치 갱신 공식

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

여기서

- w : 가중치
- L : 손실 함수
- η : 학습률(Learning Rate)
- $\frac{\partial L}{\partial w}$: 기울기

학습률(Learning Rate)

학습률은 한 번에 얼마나 이동할 것인지를 결정한다.

너무 작은 경우

0.0000001

특징

- 학습이 매우 느림
- 수렴 시간이 오래 걸림

너무 큰 경우

10

특징

- 최적점을 지나쳐 버림
- 발산 가능

적절한 경우

0.001

0.0001

특징

- 안정적으로 수렴

1. Batch Gradient Descent

전체 데이터를 한 번에 사용하여 기울기를 계산한다.

과정

전체 데이터

↓

기울기 계산

↓

가중치 수정

장점

- 안정적
- 정확한 방향 계산

단점

- 데이터가 많으면 느림
- 메모리 사용량 큼

2. Stochastic Gradient Descent (SGD)

데이터 1 개마다 가중치를 수정한다.

과정

데이터 1 개

↓

기울기 계산

↓

가중치 수정

장점

- 빠름
- 지역 최적점 탈출 가능

단점

- 진동 발생
- 학습이 불안정

3. Mini-Batch Gradient Descent

현재 가장 많이 사용되는 방식이다.

예

데이터 10000 개

배치 크기 32

이면

32 개씩

기울기 계산

가중치 수정

을 반복한다.

장점

- 속도와 안정성 균형
- GPU 활용 가능

단점

- 배치 크기 선택 필요

4. Momentum

이전 기울기 방향을 기억하여 학습 속도를 높인다.

공이 굴러가는 것과 비슷하다.

이전 방향 기억

+

현재 방향

Momentum 공식

$$v_t = \gamma v_{t-1} + \eta \nabla J$$

$$w = w - v_t$$

장점

- 진동 감소
- 빠른 수렴

단점

- 하이퍼파라미터 추가

5. Nesterov Accelerated Gradient (NAG)

Momentum 을 개선한 방식이다.

먼저 이동할 위치를 예측하고 기울기를 계산한다.

현재 위치

↓

예상 이동

↓

기울기 계산

장점

- Momentum 보다 빠름
- 더 정확한 방향 계산

6. AdaGrad

파라미터마다 다른 학습률을 적용한다.

자주 업데이트되는 가중치는

학습률 감소

드물게 업데이트되는 가중치는

학습률 유지

공식

$$w = w - \frac{\eta}{\sqrt{G + \epsilon}} g$$

장점

- 희소 데이터에 강함
- NLP 에서 활용

단점

- 학습률이 계속 감소
- 나중에는 거의 학습 안 됨

7. RMSProp

AdaGrad 의 단점을 개선하였다.

과거 기울기를 모두 저장하지 않고 최근 기울기만 반영한다.

공식

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

$$w = w - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

장점

- 안정적
- RNN 학습에 효과적

단점

- 최적점 근처 진동 가능

8. Adam (Adaptive Moment Estimation)

현재 가장 많이 사용하는 최적화 알고리즘이다.

Momentum 과 RMSProp 을 결합하였다.

구성

Momentum + RMSProp

Adam 의 핵심 공식

1차 모멘트(평균)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2차 모멘트(분산)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

가중치 갱신

$$w = w - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

기본 설정값

```
learning_rate = 0.001  
beta1 = 0.9  
beta2 = 0.999  
eps = 1e-8
```

장점

- 빠른 수렴
- 자동 학습률 조정
- 대부분의 문제에서 우수

단점

- 메모리 사용량 증가
- 일반화 성능이 SGD 보다 떨어질 수 있음

9. AdamW

Adam 의 Weight Decay 문제를 개선한 버전이다.

최근 딥러닝 모델에서 가장 많이 사용된다.

예

- Transformer
- BERT
- GPT
- Vision Transformer

등

장점

- 과적합 감소
- 일반화 성능 향상

10. Lion

Google 이 제안한 최신 Optimizer 중 하나이다.

Adam 보다 메모리 사용량이 적다.

특징

Adam

↓

더 적은 메모리

↓

더 빠른 학습

대규모 언어모델 학습에 활용된다.

Optimizer 비교

Optimizer	속도	안정성	메모리	현재 활용도
SGD	보통	높음	적음	높음
Momentum	빠름	높음	적음	높음
NAG	빠름	높음	적음	보통
AdaGrad	보통	높음	보통	낮음
RMSProp	빠름	높음	보통	보통
Adam	매우 빠름	매우 높음	많음	매우 높음
AdamW	매우 빠름	매우 높음	많음	최고
Lion	매우 빠름	높음	적음	증가 중

최적화 함수는 역전파를 통해 계산된 기울기를 이용하여 신경망의 가중치를 수정하는 알고리즘이다. 가장 기본은 경사하강법(SGD)이며, 이를 개선한 Momentum, NAG, AdaGrad, RMSProp 이 등장하였다.

현재는 RMSProp 의 적응형 학습률과 Momentum 의 이동 평균 개념을 결합한 Adam 이 널리 사용되며, Transformer 계열 모델에서는 AdamW 가 사실상 표준으로 사용된다.

실무에서는 대부분 **Adam** 또는 **AdamW** 로 시작하고, 이미지 분류나 일반화 성능이 중요한 경우에는 **SGD + Momentum** 도 많이 사용된다.

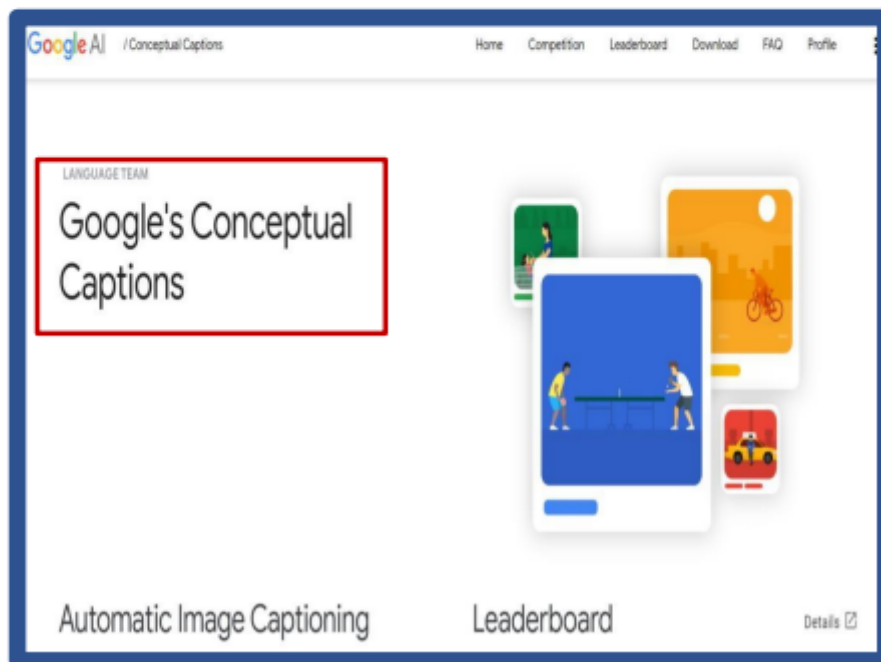
3. 딥러닝 RNN 모델

RNN 으로 항공 여행자 수 예측하기

1) 자동 이미지 캡션 처리 대회 (구글 주최)

(1) 3백3십만 개의 사진에 자동 캡션을 다는 인공지능 기술 경쟁

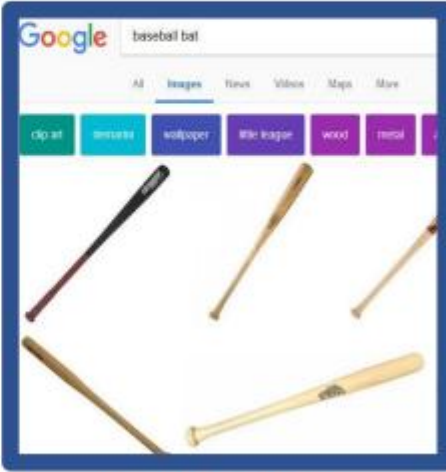
- 비전: CNN을 이용해 이미지 인식
- 언어: RNN을 이용해 추출된 단어들을 문장으로 전환



QUESTION



baby



baseball bat

9/12/2014

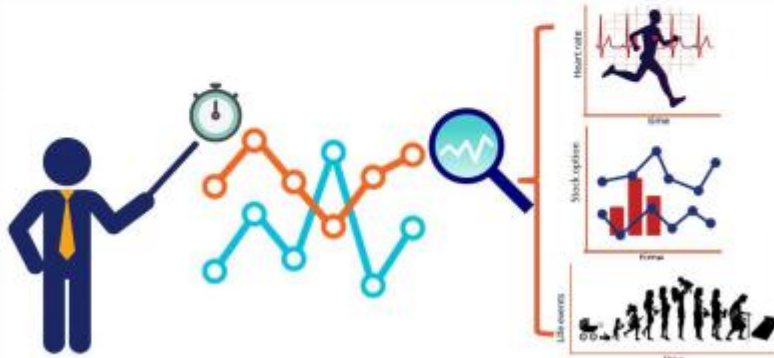


newborn baby holding baseball bat

1) 순환 신경망(Recurrent Neural Network)

(1) 순환 신경망(Recurrent Neural Network)

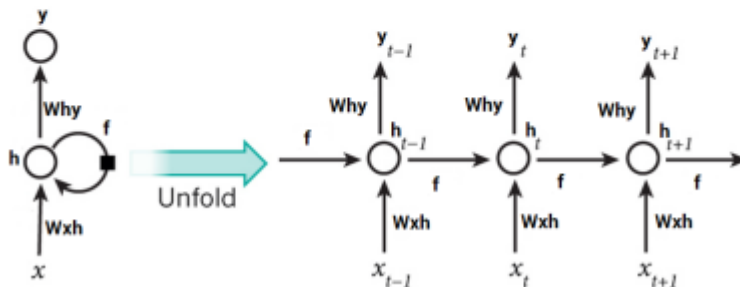
- David Rumelhart (1986)
- 시계열 데이터를 학습하는 딥러닝 기술
- 기준 시점 (t)과 다음 시점 (t+1)에 네트워크를 연결
- AI 번역, 음성 인식, 주가 예측의 대표적 기술



2) RNN 풀어 헤침 (unfolding)

(1) RNN 풀어 헤침(unfolding)

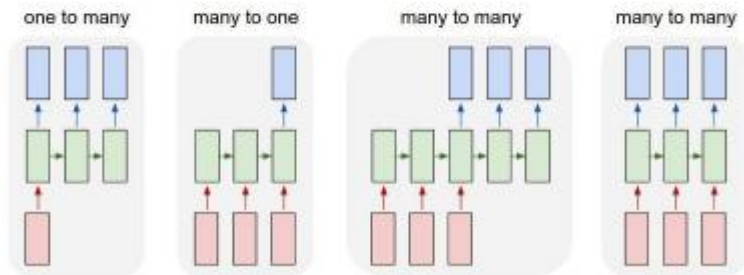
- RNN 마스터 구조 사용
- 입력 데이터의 숫자 만큼 풀어 헤침 (unfold)



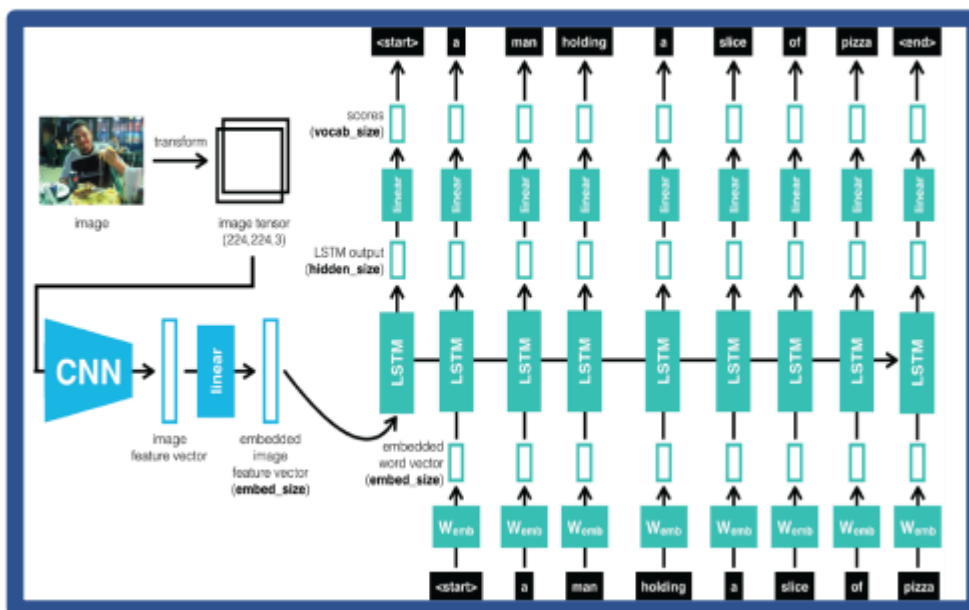
3) RNN 종류

(1) 다루는 문제에 따라 다름

- One-to-many: 한 음으로 다음 여러 개의 음을 작곡
- Many-to-one: 여러 입력을 통해 하나의 출력값 생성 (이번 RNN에서 사용)
- Many-to-many: 번역기와 챗봇에 주로 사용



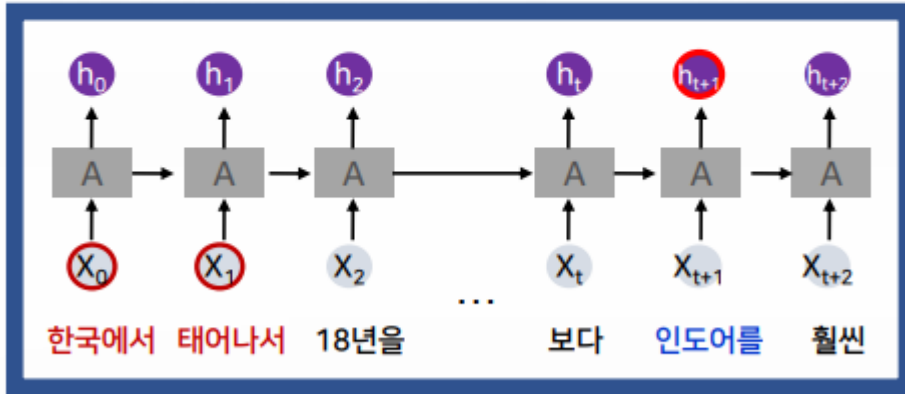
6) CNN-RNN 아키텍처 및 학습 과정



1) LSTM 기술 소개

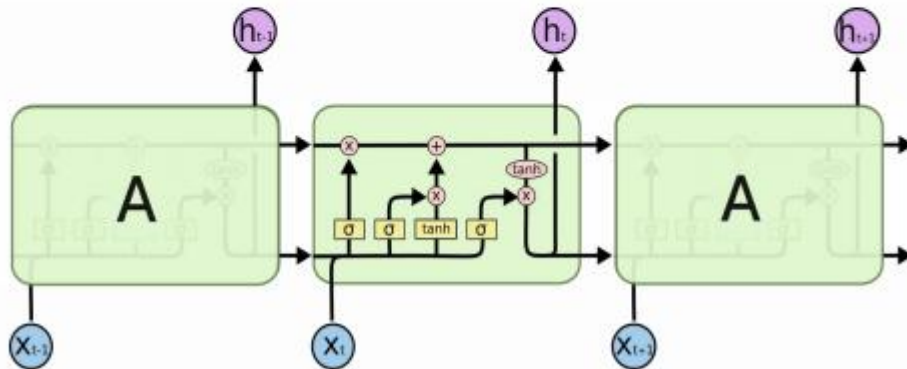
(1) 기울기 유실 문제 (vanishing gradient)

- 옛날 일을 기억하지 못함
- 오래 전의 데이터에 의한 기울기 값이 소실되는 문제



(2) Hochreiter & Schmidhuber (1997)

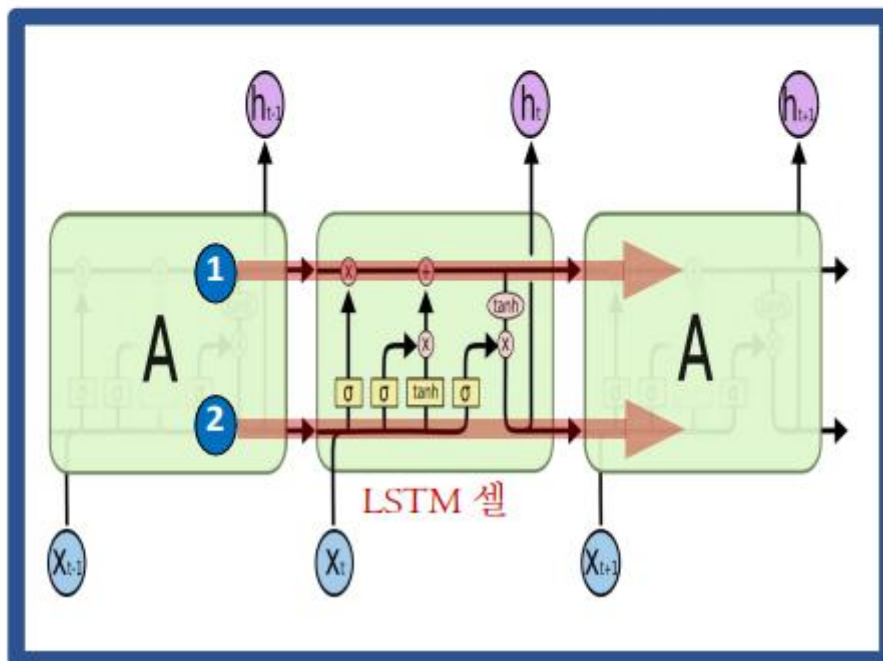
- RNN의 기울기 유실 문제 해결
- 2가지 종류의 정보를 다음 단계 전달



2) 장단기 메모리(Long-Short Term Memory)

(1) Hochreiter & Schmidhuber (1997)

- RNN의 기울기 유실 문제 해결
- 2가지 종류의 정보를 다음 단계 전달

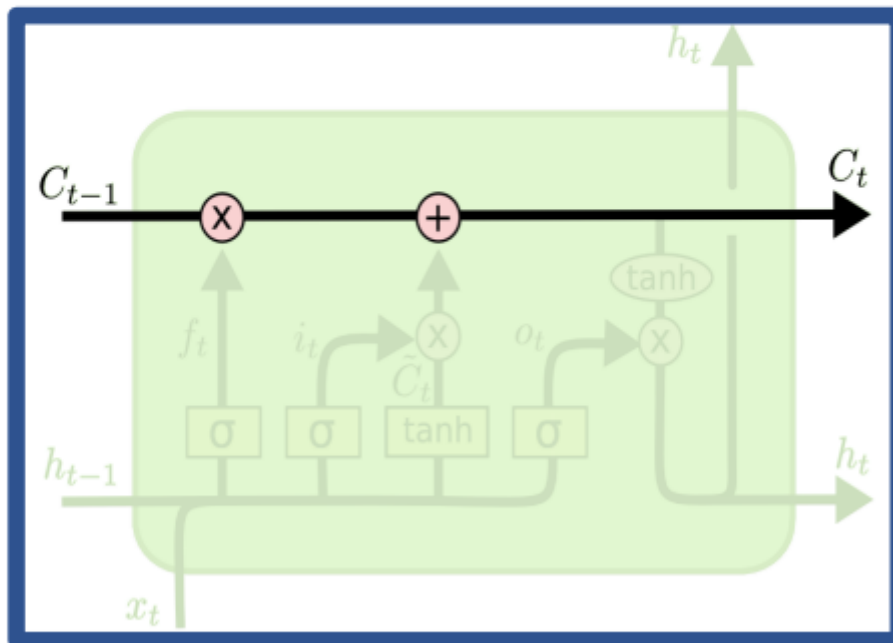


3) LSTM : Cell state 개념

(1) 하나의 컨베이어 벨트처럼 전체 체인을 통과

- 현 단계 정보를 수정하여 계속 다음 단계에 전달
- 게이트 (gate)를 활용해서 정보를 더하거나 제거

제재나 가감 없으면 정확도가 떨어질 수 있음



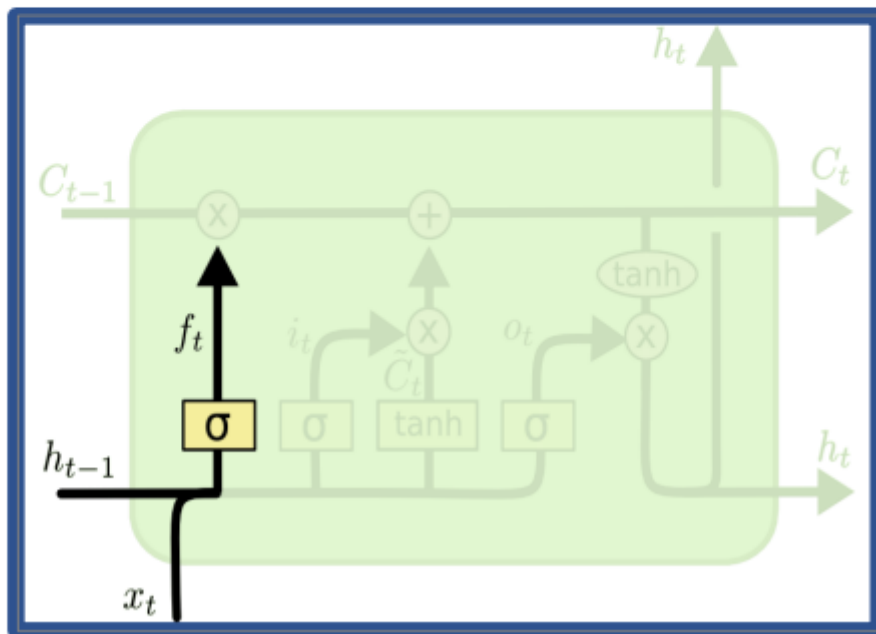
셀스테이트 라인 자체에 활성화 함수 없음

➡ 베네싱 그래디언트 문제에서 해방됨

4) LSTM Gate 1 : 망각 Gate

(1) Cell state 라인에 추가/삭제 양을 조절

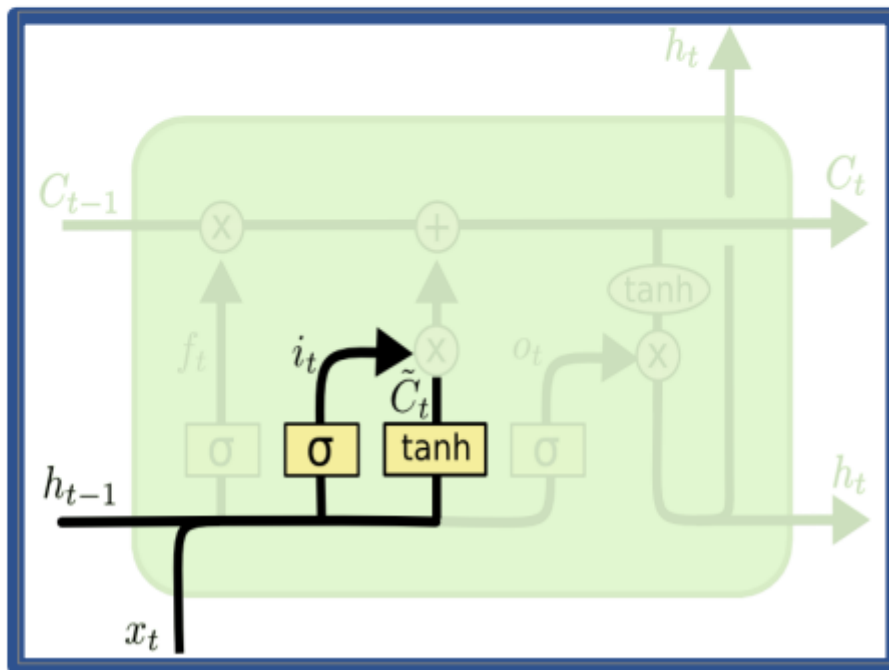
- 망각 Gate: 전 단계에서 오는 정보를 얼마나 잊어 버릴까 결정, $[0, 1]$



5) LSTM Gate 2 : 새 정보 Gate

(1) Cell state 라인에 추가/삭제 양을 조절

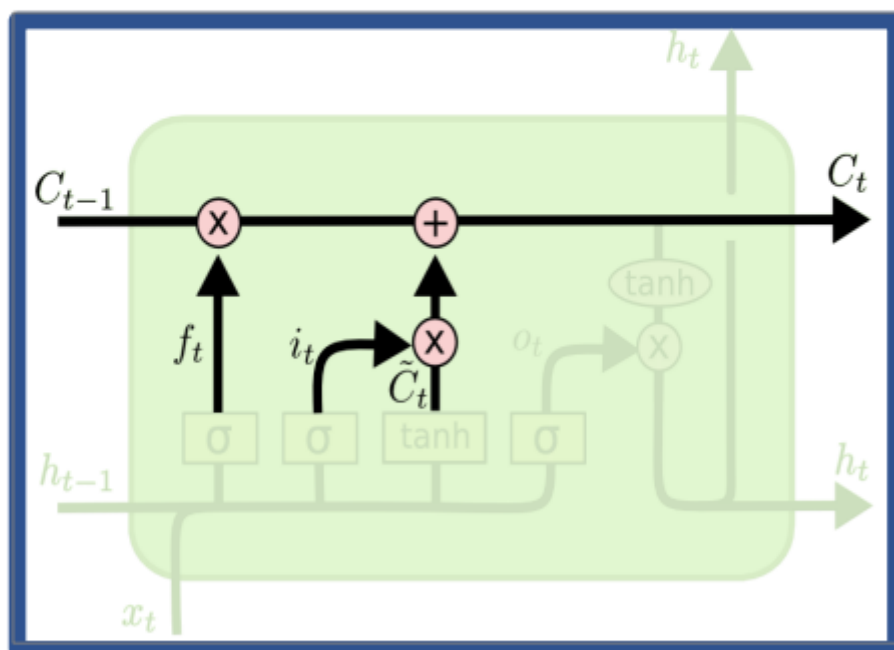
- 새 정보 Gate: 새로 들어오는 입력값을 얼마나 받아 들일까 결정, $[-1, 1]$



6) LSTM Gate 3 : Cell State 출력 Gate

(1) Cell state 라인에 추가/삭제 양을 조절

- Cell State 출력 Gate: 다음 단계 전해질 cell state의 양을 결정



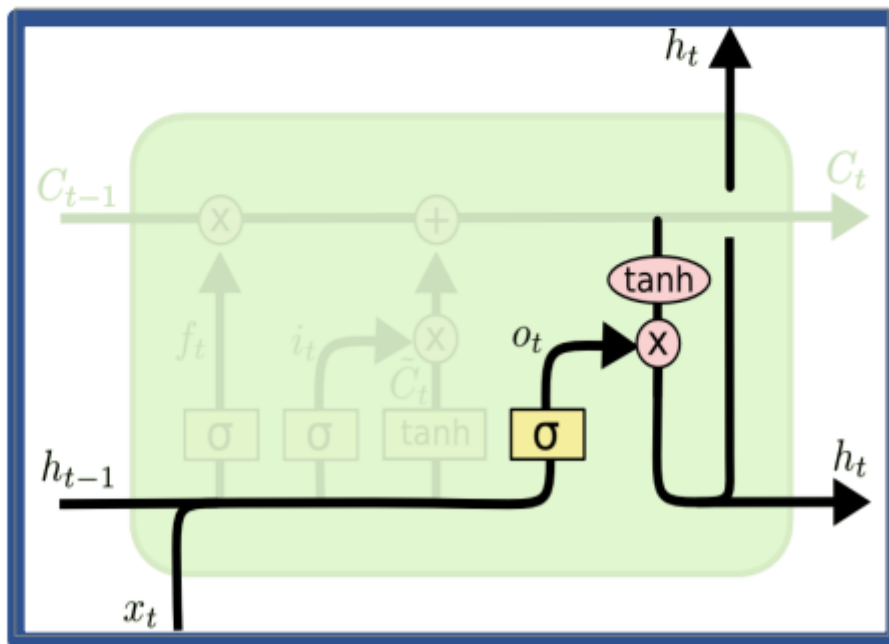
셀스테이트 출력 위한 활성화 함수 사용 안함

- ➡ 망각과 새정보 게이트어떻게 합성을 할 것인지 결정하는 중치가 존재하지 않음

7) LSTM Gate 4 : 현재 단 출력 Gate

(1) Cell state 라인에 추가/삭제 양을 조절

- 현재 단 출력 Gate: 현재 단의 출력을 결정



1) 1950년대 비행기 여행

(1) 비행기 여행의 황금기

- 전 좌석이 일등석, 승객들 모두 정장, 비용은 거의 5배



2) Air Passengers Dataset

(1) Kaggle.com

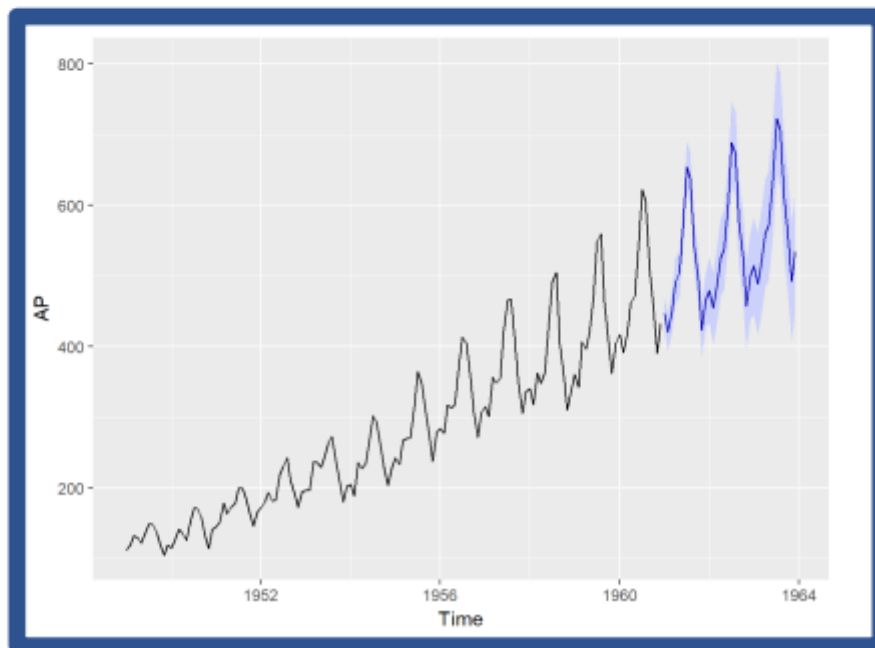
- “air passengers dataset”으로 검색
- 파일 이름: AirPassengers.csv (CSV 형식)
- 매달 미국 국내/해외 비행기 여행자수 (1,000명 단위)
- 1949년1월부터 1960년12월까지 (144개의 데이터)



3) Air Passengers Dataset : 시계열 데이터

(1) 각 데이터 값이 시간의 개념과 연동

- 트렌드를 분석하고 미래를 예측하는데 사용



1. RNN 소개

- 자동 이미지 캡션 기술
 - CNN을 통해 인식 후 언어(RNN) 완성함
- 순환 신경망 이론
 - RNN은 시계열 데이터를 학습하는 기계학습 기술임
 - 마스터 구조를 입력 데이터의 숫자 만큼 풀어 헤침 (unfold)

2. LSTM 이론

- LSTM 기술 소개
 - RNN의 단점인 과거 입력을 잊어버리는 단점을 개선함
- LSTM cell state 개념
 - 하나의 컨베이어 벨트처럼 전체 체인을 통과하는 연결 라인 구축함
 - 현 단계 정보를 수정하여 계속 다음 단계에 전달함
- LSTM gating 이론
 - 게이트(gate)를 활용해서 정보를 더하거나 제거하는 기능을 수행함

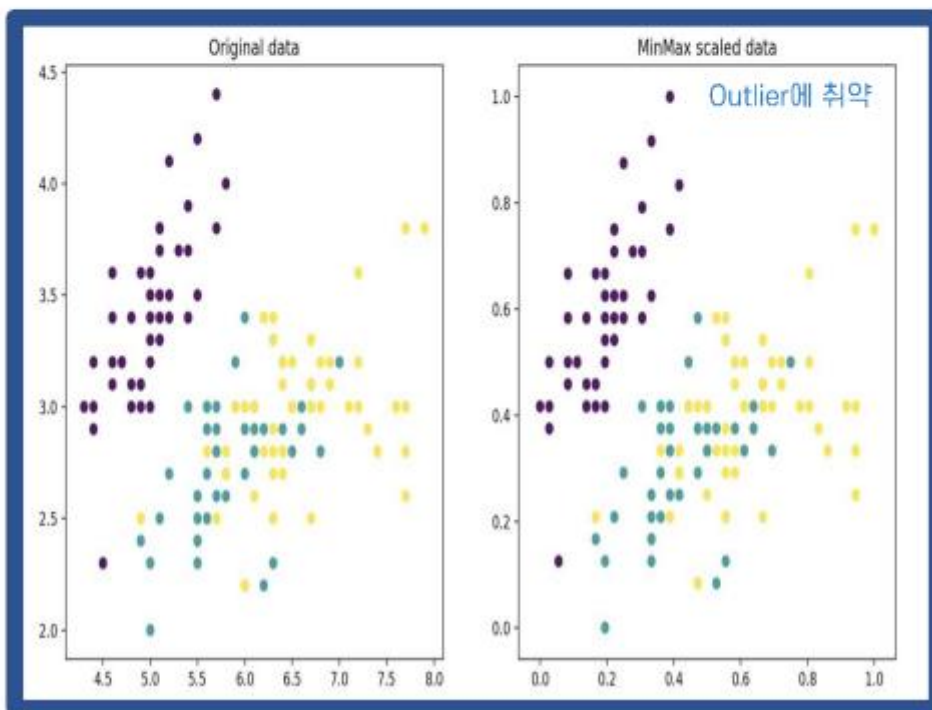
3. Dataset 소개

- Airline dataset 소개
 - 1949년 1월부터 1960년 12월까지 매달 미국 국내/해외 비행기
여행자 수의 데이터임
 - 144개의 데이터임
- 시계열 데이터 개념
 - 각 데이터 값이 시간의 개념과 연동함
 - 트렌드를 분석하고 미래를 예측하는데 사용됨

1) MinMax 데이터 정규화

(3) [0, 1] 스케일링

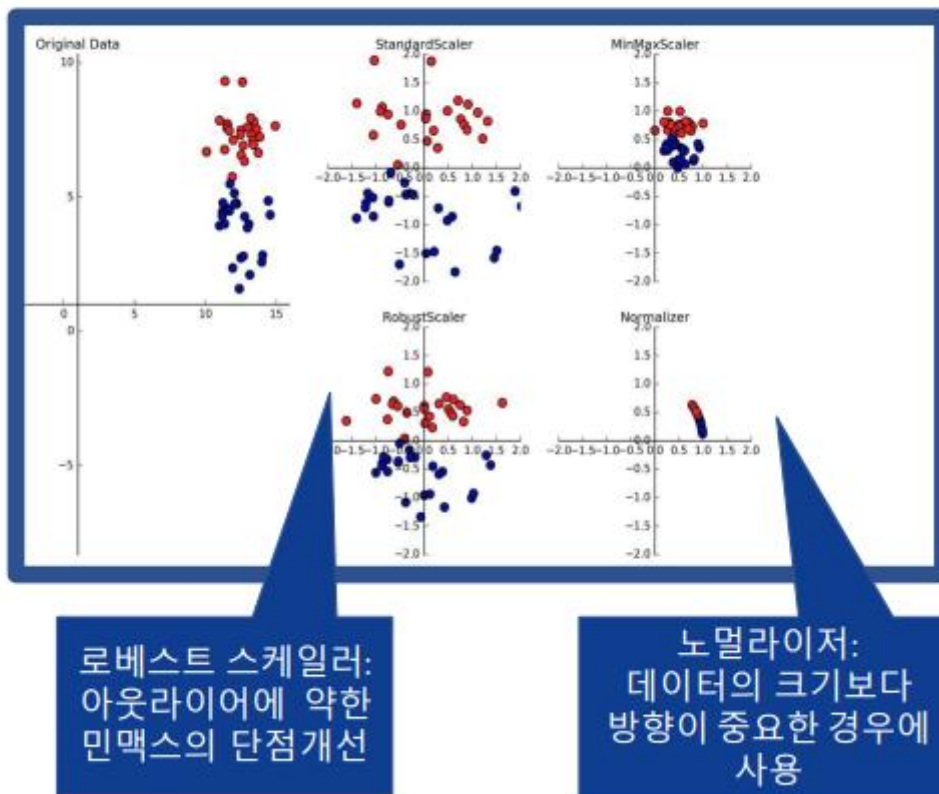
$$x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}$$



- MinMax 정규화 이론
 - [0,1] 스케일링 : 모든 데이터값을 0,1 범위로 전환함
 - 로베스트 스케일러: 아웃라이어에 약한 민맥스의 단점 개선함
 - 노멀라이저: 데이터의 크기보다 방향이 중요한 경우에 사용함
- 정규화 데이터 코딩
 - Z-점수 정규화와 비교 분석 진행
 - Pandas의 head와 describe 함수로 데이터 내용 확인

1) MinMax 데이터 정규화

(4) 정규화의 종류



1) Inverse Transformation 이론

(2) MinMax 정규화 공식

$$x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

(3) MinMax 역전환 공식

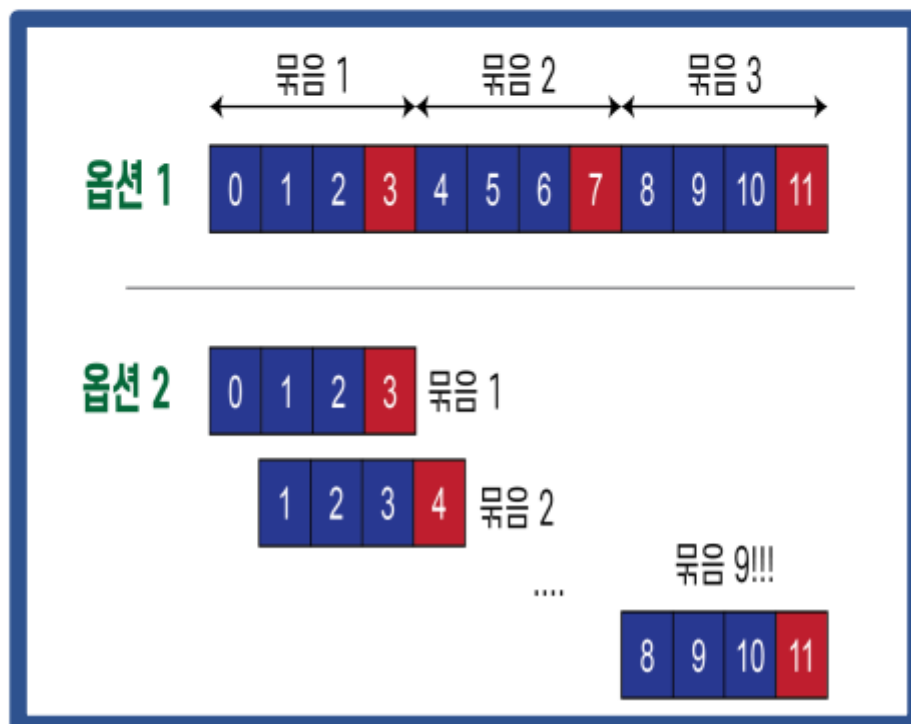
$$x = x_{scaled} (\max(x) - \min(x)) + \min(x)$$

- LSTM은 [0, 1] 정규화데이터로 학습
- LSTM predict 결과는 [0, 1] 정규화됨
- Scikit-learn의 `inverse_transform` 함수 사용
- [0, 1] 역전환으로 원래 여행자 수 복귀

1) 시계열 데이터 분할 묶음

(1) Air Passenger Dataset: 144개 데이터

- 12달치는 입력, 13번째는 출력
- 데이터 분할 필요: 두가지 옵션 존재

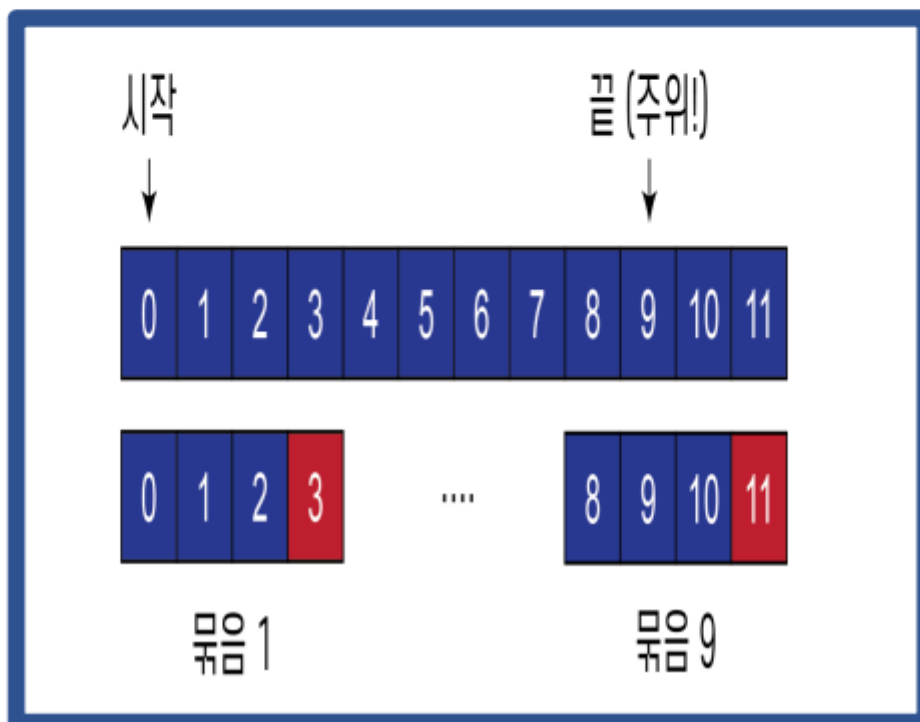


- 분할 묶음 코딩
 - 파이썬 리스트를 사용하여 시계열 데이터를 13묶음으로 분할함
- 데이터 사분할
 - Numpy를 사용하여 데이터를 입력과 출력으로 2분할함
 - Numpy를 사용하여 데이터를 학습용과 평가용으로 2분할함

1) 시계열 데이터 분할 묶음

(2) 파이썬 리스트 사용

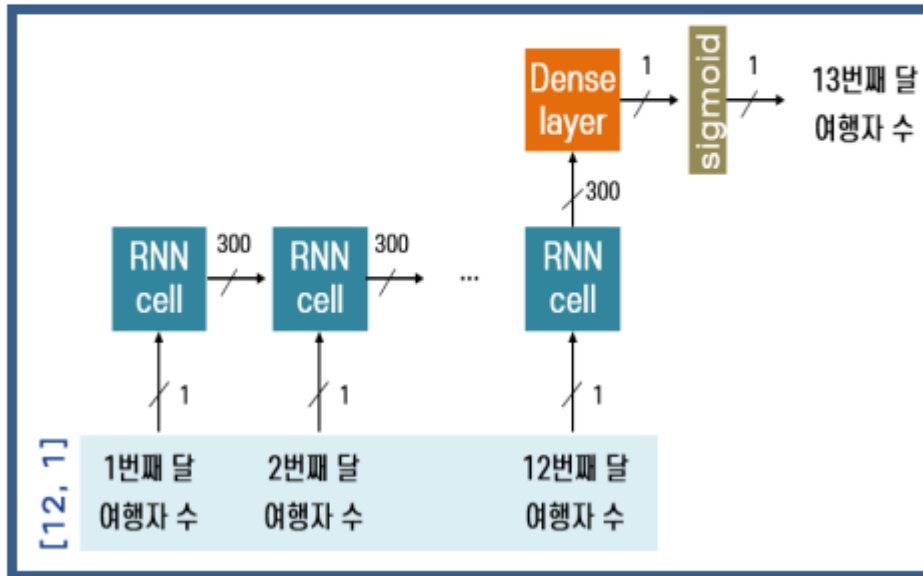
- 3: 입력 수, 12: 데이터 수
- 총 묶음 수: $12 - 3 + 2 = 9$
- 마지막 9번째 묶음: 8, 9, 10, 11
- For-loop 범위: 0에서 12-3 (8까지만 방문!)
- 분할 범위: i에서 i+3+1 (i+3까지만 처리!)



1) RNN 설계도

(1) 13개 묶음의 시계열 데이터 처리

- 입력: 12달 치 여행자 수
- 출력: 13번째 달 여행자 수 예측



- RNN 설계도
 - 13개 묶음의 시계열 데이터 처리함
 - 입력: 12달 치 여행자 수
 - 출력: 13번째 달 여행자 수 예측
- RNN 구현
 - Keras Sequential 모듈을 활용하여 LSTM 구현함
 - LSTM 셀의 차원을 300으로 지정함
 - 출력값은 Sigmoid 활성화 함수를 활용함
 - [0, 1] MinMax 정규화된 입력 데이터와 맞춤
- Sigmoid 활성화 함수
 - 장점: 미분 계산이 간단함
 - 단점: 기울기 유실 문제가 있음

2) Numpy Flatten 함수

(1) LSTM평가와 관련된 데이터 모양

- X_test (평가용 입력 데이터): numpy 3차원 행렬
- Y_test (평가용 출력 데이터): numpy 2차원 행렬
- Pred (LSTM 예측값): numpy 2차원 행렬

(2) LSTM예측 결과의 시각화

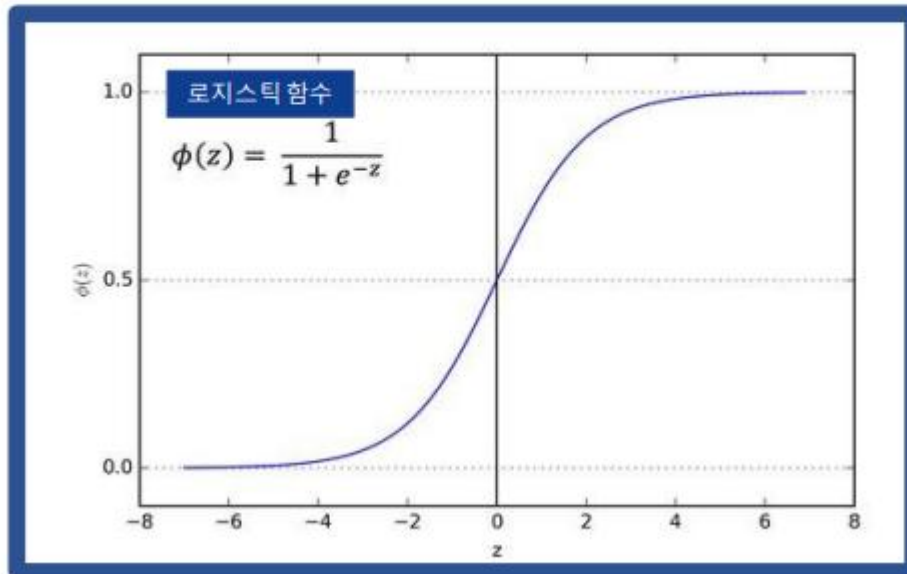
- Seaborn package의 lineplot 사용 예정
- Lineplot 함수는 1차원 행렬 요구

(3) Numpy flatten 함수

- Numpyn-차원 행렬을 1차원으로 축소

2) Sigmoid 활성화함수

(1) 딥러닝에서가장많이 쓰이는활성화함수



2) Sigmoid 활성화함수

(1) 딥러닝에서가장많이 쓰이는활성화함수

- Step function을 대체: 전 구간에서 미분가능
- 장점: 미분 계산이간단

$$s'(x) = s(x)\{1 - s(x)\}$$

- 단점: 기울기 유실 문제

1) Rmsprop 최적화알고리즘

(1) Root Mean Square Propagation

- 제프리 힌턴 교수팀 제안



역전파 알고리즘
(백 프로퍼게이션
알고리즘)

- 학습률을 상황에 맞게 조절
(시냅스의 가중치를 보정할때 정하는 숫자)

1) Rmsprop 최적화알고리즘

(1) Root Mean Square Propagation

- 진동을 줄이는 효과

For each Parameter w^j

(j subscript dropped for clarity)

$$\nu_t = \rho \nu_{t-1} + (1 - \rho) * g_t^2$$

$$\Delta \omega_t = -\frac{\eta}{\sqrt{\nu_t} + \epsilon} * g_t$$

$$\omega_{t+1} = \omega_t + \Delta \omega_t$$

η : Initial Learning rate

ν_t : Exponential Average of squares of gradients

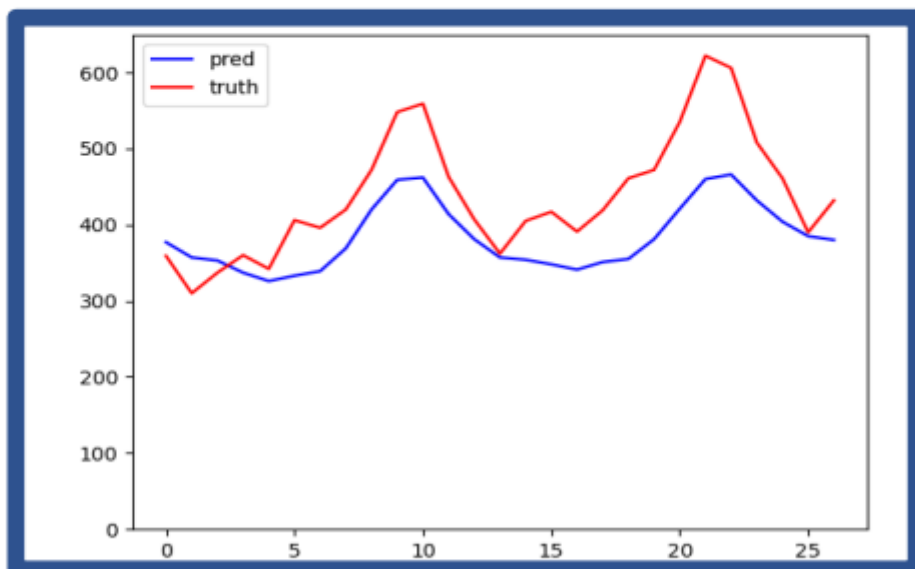
g_t : Gradient at time t along w^j

- RMSprop 최적화 알고리즘 이론
 - 학습률을 최적화 상황에 맞게 조절해 가며 운용함
- 인공 신경망 학습 코딩
 - Keras RMSprop 최적화 알고리즘 활용함
 - MSE 손실 함수 학습 및 활용함

3) Seaborn Lineplot 함수

(1) 선형 그래프 생성

- LSTM 정확도 평가: MSE 손실값 숫자 하나로는 부족
- LSTM의 예측값과 dataset의 정답을 시각적으로 비교



- 인공 신경망 평가코딩
 - Scikit-learn package의 inverse transform 함수를 활용함
 - Keras의 predict 함수를 사용하여 RNN 평가 진행함
- Inverse Transformation 이론
 - MinMax 역전환 공식을 통해 $[0,1]$ 역전환으로 원래 여행자 수
복귀함
- Seaborn Lineplot 함수
 - 예측값과 정답의 차이를 시각화함

4. 딥러닝 GAN 모델

DNN-GAN 으로 손글씨 모방하기

1) GAN: 생성된 가상인물

(1) 실존 인물이 아닌 창작품



- 이미지 창작
 - 기존에 존재하지 않는 새로운 인물의 사진을 감쪽같이 생성
 - 기존에 존재하지 않는 새로운 패션아이템을 스케치만으로 이미지화함
- 문장에서 이미지로 전환
 - 이미지에서 문장 전환보다 훨씬 어려움
- 그 외에 이미지를 대상으로 한 여러가지 창조기술을 구사

2) GAN : 스케치를 시제품으로

(1) 시제품이 실물과 비교해 손색이 없음



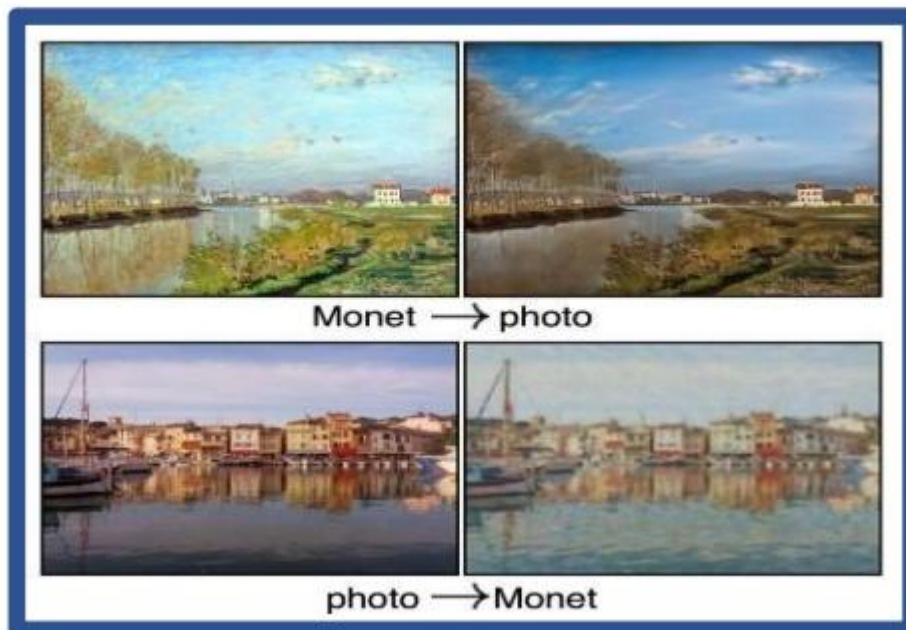
2) GAN : 사진 찍다 눈을 감았다면?

(1) 데이터가 많으면 많을 수록 효과 증가



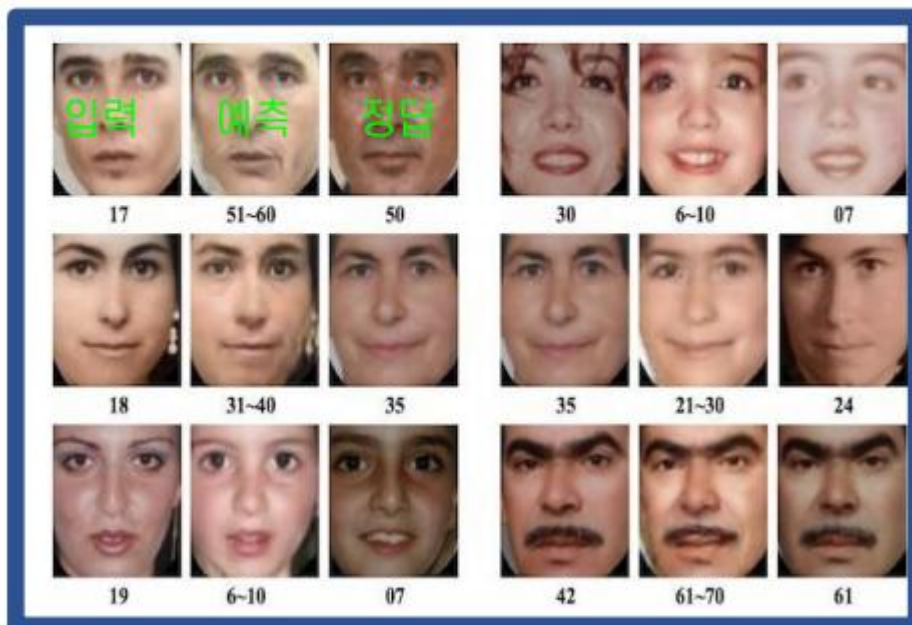
4) GAN : 도메인 전환

(1) 회화와 사진의 두 영역 간의 전환



5) GAN : 노화예측

(1) 회화와 사진의 두 영역 간의 전환



6) 자동 이미지 캡션 처리 기술

(1) 비전 (CNN)과 언어 (RNN)가 결합된 인공지능



7) GAN : 문장에서 이미지로 전환

(1) 이미지에서 문장 전환보다 훨씬 어려움



8) GAN : 그 다음은? 영상 모방/창조!

(1) Video to Video Synthesis



1) GAN : Generative Adversarial Network

(1) 이안 굿펠로우: GAN의 창시자(2014)



(2) 얀 르쿤 교수



“

최근 20년간

가장 훌륭한 기계학습 기술!

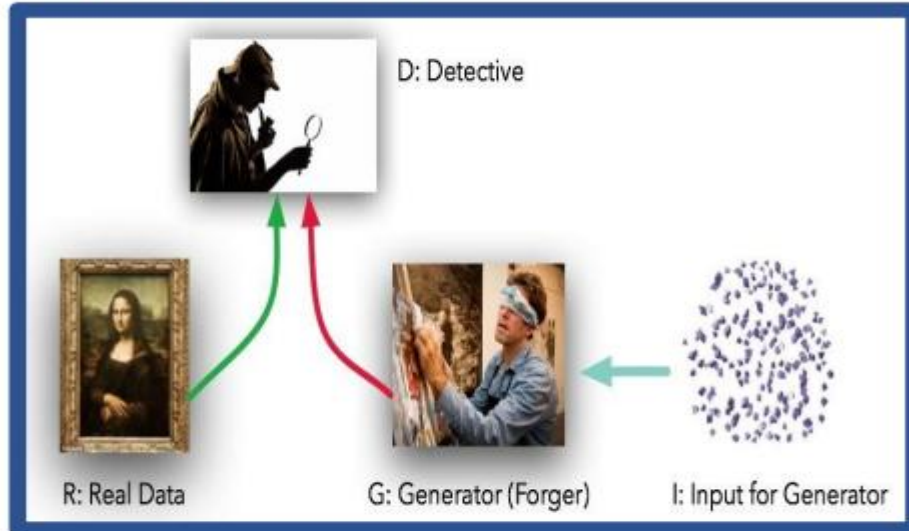
”

- GAN의 창시자
 - 2014년 이안 굿펠로우가 창시함
- GAN 기술이론
 - GAN은 생성자와 감별자, 두개의 인공신경망을 사용함
 - 생성자와 감별자가 서로 경쟁하며 똑똑해짐
 - 생성자는 감별자를 속일 목적으로 훈련
 - 감별자는 생성자의 가짜 작품을 가려낼 목적으로 훈련

2) GAN의 원리

(1) 두 개의 인공 신경망이 서로 경쟁하며 학습하는 방식

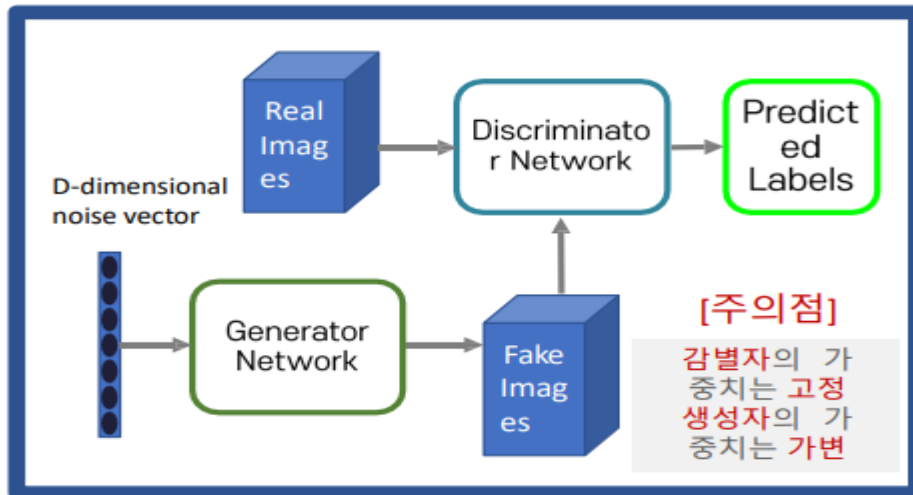
- **생성자(Generator)**: 실제 데이터를 학습하고 이를 바탕으로 거짓 데이터를 생성
- **감별자(Discriminator)**: 생성자가 내놓은 거짓 데이터가 거짓임을 판별하도록 학습



3) 생성자학습방식

(1) 한 번의 학습을거침

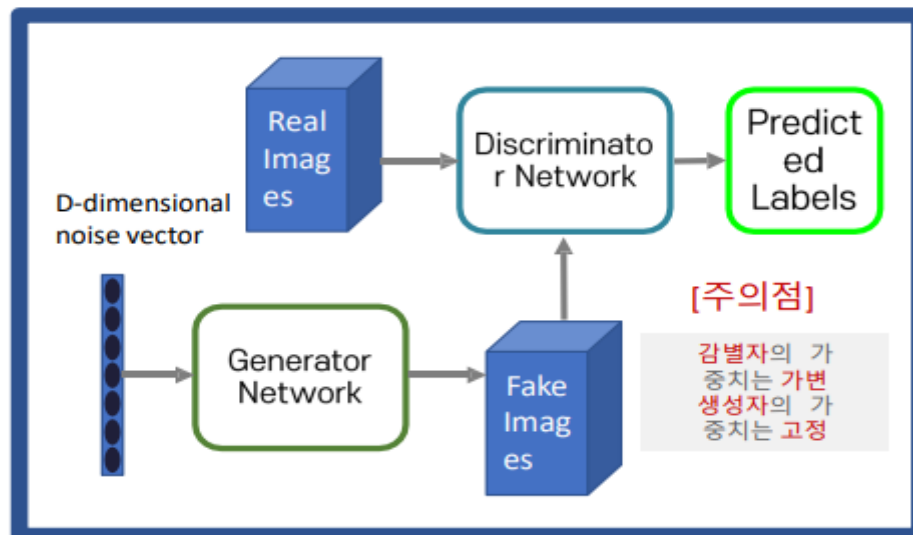
- 생성자가가짜 이미지 하나 생성해서 감별자에게 줌
- 감별자가가짜라고 감별하면 실패: 생성자가중치 보정
- 감별자가진짜라고 속으면 성공



4) 감별자학습방식

(1) 두 번의 학습을거침

- 생성자가가짜 이미지 하나 생성
- 이 **가짜** 이미지를 감별자에게: **가짜**로 판별하도록 학습
- 진품 참고에서 진짜 이미지 하나 임의로 추출
- 이 **진짜** 이미지를 감별자에게: **진짜**로 판별하도록 학습



5) GAN의 역전파알고리즘

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{data}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log (1 - D(G(\mathbf{z}^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(\mathbf{z}^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

6) 생성자 손실 함수

(1) 이 함수를 **최소화** 하자!

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - \underbrace{D \left(G \left(z^{(i)} \right) \right)}_{\text{최대화!}} \right)$$

- $D(G(z(i)))$: 감별자가 i 번째 가짜를 감별한 결과: 0은 가짜, 1은 진짜
- 생성자는 감별자를 속이기 위해 학습
- 왜 log? 확률 변수를 다루기 용이
- $\text{Log}(x \cdot y) = \log(x) + \log(y)$
- ∇ (편미분): 기울기를 계산해서 경사하강법으로 보정

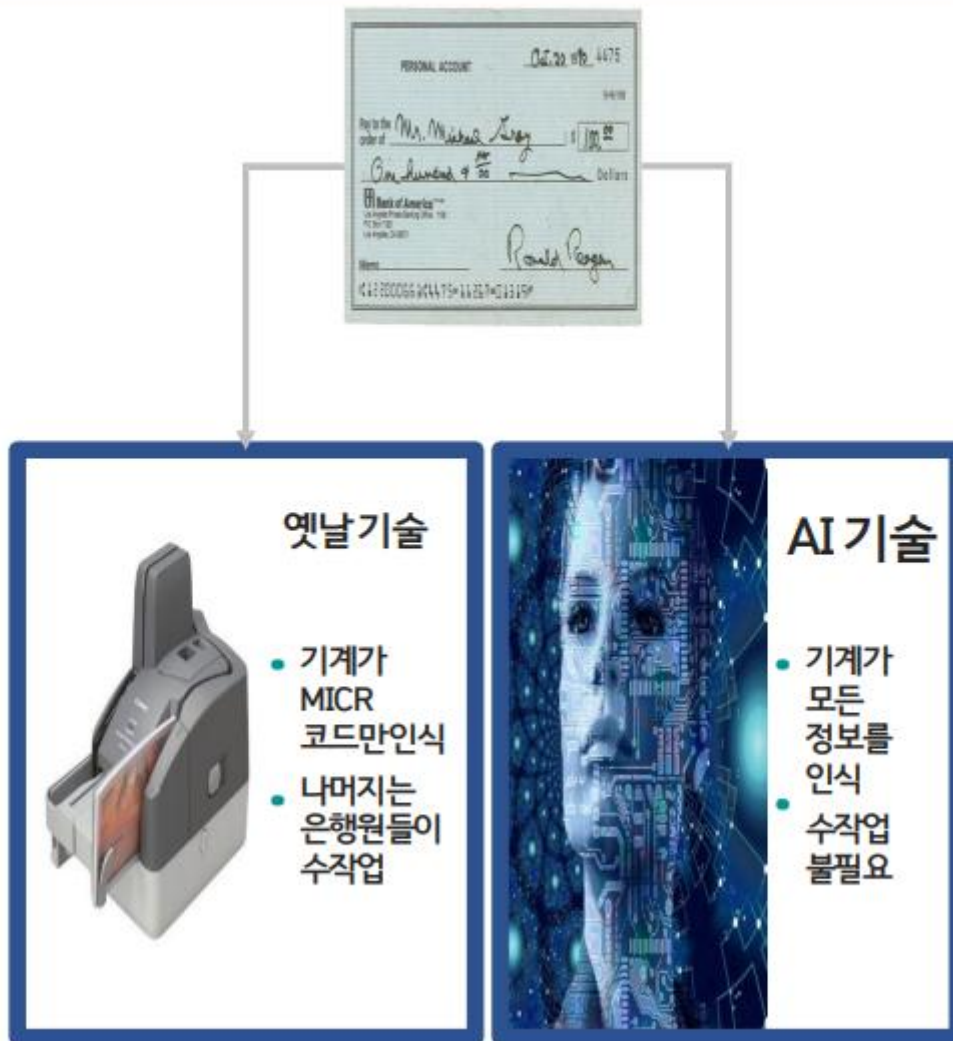
7) 감별자 손실 함수

(1) 이 함수를 **최대화** 하자!

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\underbrace{\log D \left(x^{(i)} \right)}_{\text{최대화!}} + \log \left(1 - \underbrace{D \left(G \left(z^{(i)} \right) \right)}_{\text{최소화!}} \right) \right]$$

- $D(x(i))$: 감별자가 i 번째 진짜를 감별한 결과: 0은 가짜, 1은 진짜
- $D(G(z(i)))$: 감별자가 i 번째 가짜를 감별한 결과: 0은 가짜, 1은 진짜
- 진짜를 보고 판단한 확률은 1에 가까워야 함
- 가짜를 보고 판단한 확률은 0에 가까워야 함

1) 손글씨 판별 기술

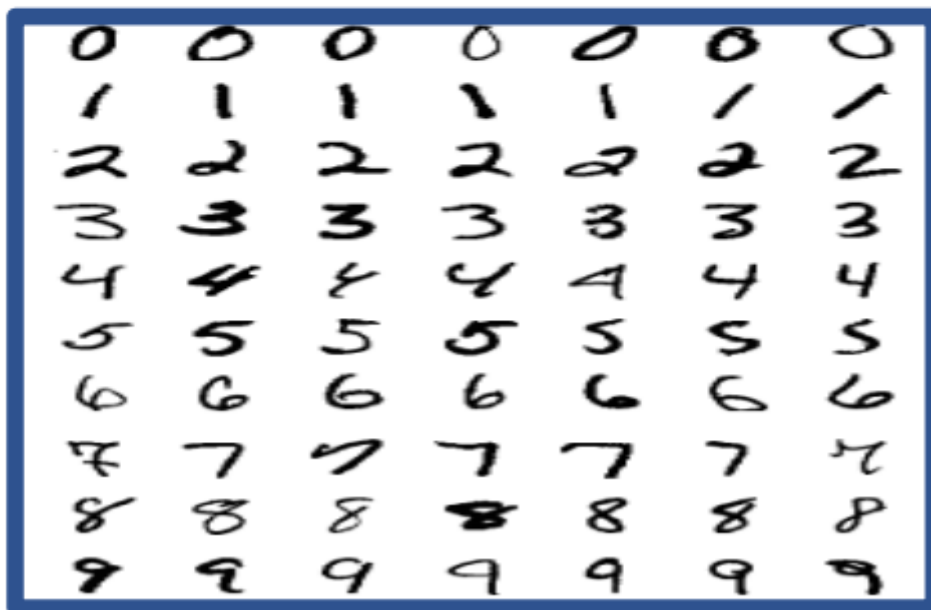


- 손글씨 판별 기술
 - 기계가 모든 정보를 인식하여 수작업 불필요함
- MNIST 손글씨 Dataset
 - MNIST는 미국 인구조사때 수집한 손글씨 데이터
 - 인공지능기술 개발에 가장 많이 사용된 dataset
 - 28x28 해상도의 흑백 이미지 7만개로 구성

2) MNIST 손글씨 숫자 Dataset

(1) 기계학습에(가장)많이 사용되는 Dataset

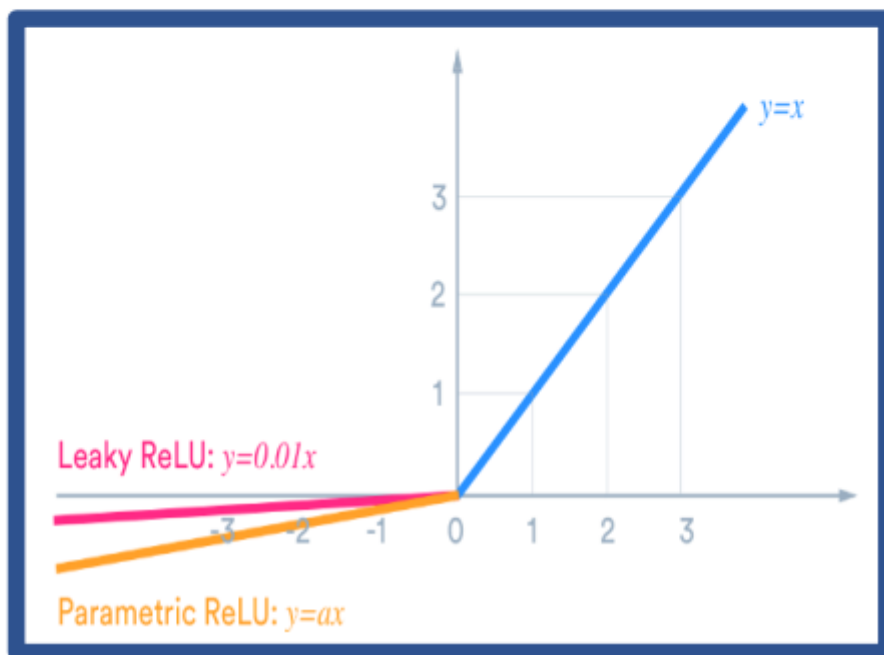
- 미국 인구조사에 쓰인 손글씨
- NIST (미국 국립 표준기술연구소)에서 공개
- 6만개의 학습용 데이터, 1만개의 평가용 데이터 존재
- 흑백 이미지 데이터
- 화상도는 28x28



1) Leaky ReLU 활성화 함수

(1) ReLU 함수에서 파생

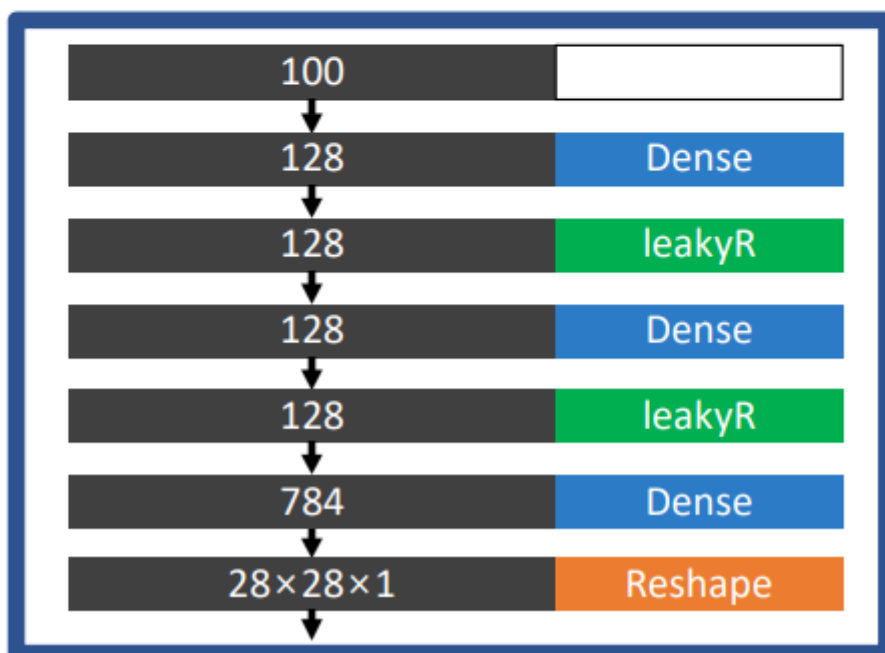
- ReLU의 단점인 죽는 뉴런을 방지하는 효과
- 기울기 유실 문제 부분적 해결
- 음수 쪽의 기울기 조정 가능



2) 생성자 설계도

(1) 감별자를 속이는 것이 목적

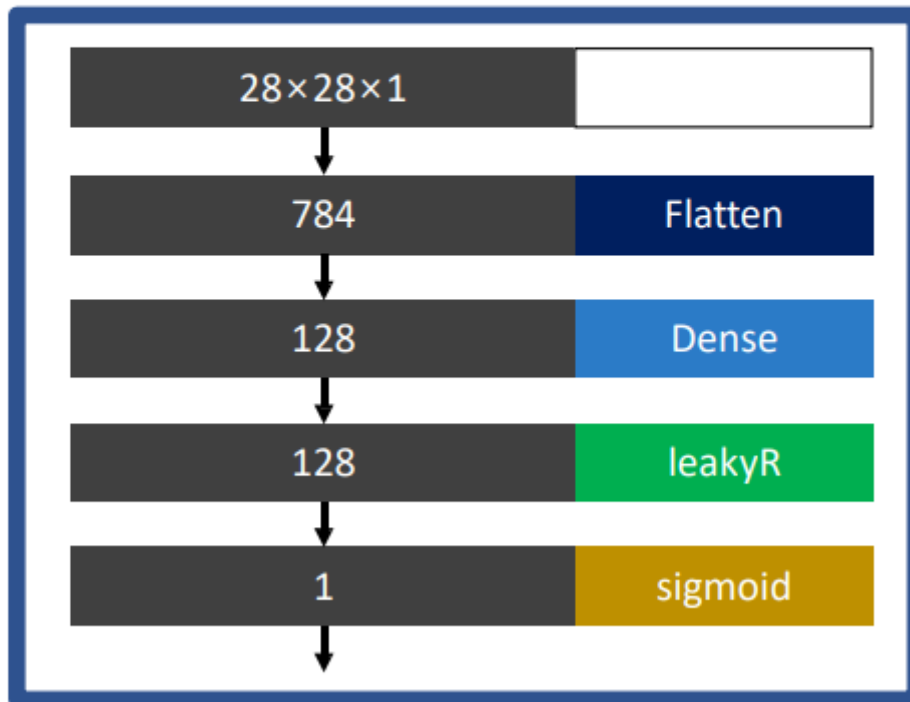
- 입력: 노이즈 벡터, 출력: 가짜 이미지 하나



3) 감별자 설계도

(1) 생성자의 가짜 이미지를 감별하는 것이 목적

- 입력: 이미지(가짜/진짜), 출력: 감별 결과 (확률)

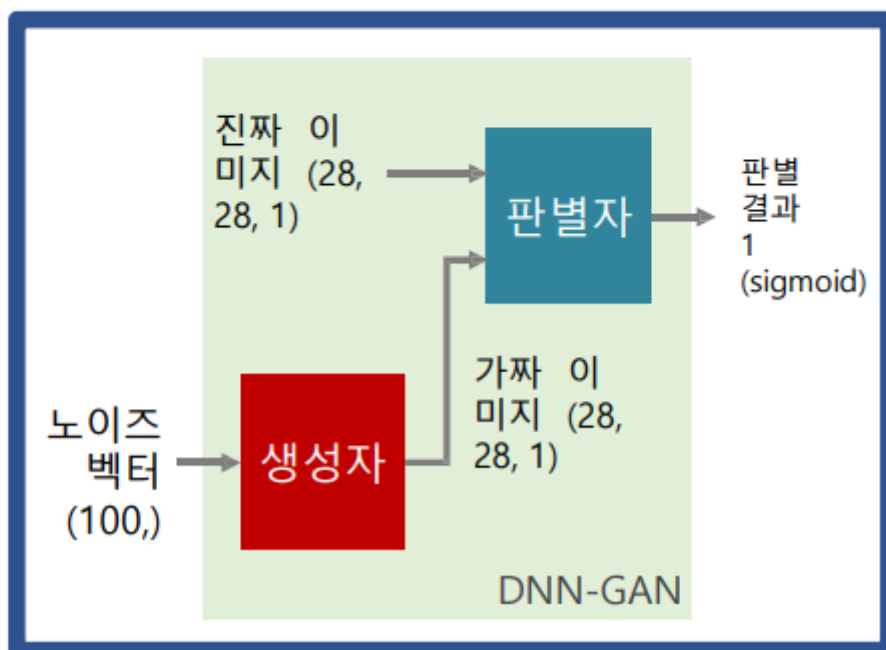


- Leaky ReLU 활성화 함수
 - ReLU의 단점인 죽는 뉴런을 방지하는 효과가 있음
 - 기울기 유실 문제 부분적 해결함
 - 음수쪽의 기울기 조정 가능함
- 생성자 구현
 - 생성자는 감별자를 속이는 것이 목적임
 - 생성자 입력: 노이즈 벡터
 - 생성자 출력: 가짜 이미지 하나
- 감별자 구현
 - 감별자는 생성자의 가짜 이미지를 감별하는 것이 목적임
 - 감별자 입력: 이미지(가짜/진짜)
 - 감별자 출력: 감별 결과(확률)

1) DNN-GAN 설계도

(1) 생성자와 감별자 연결

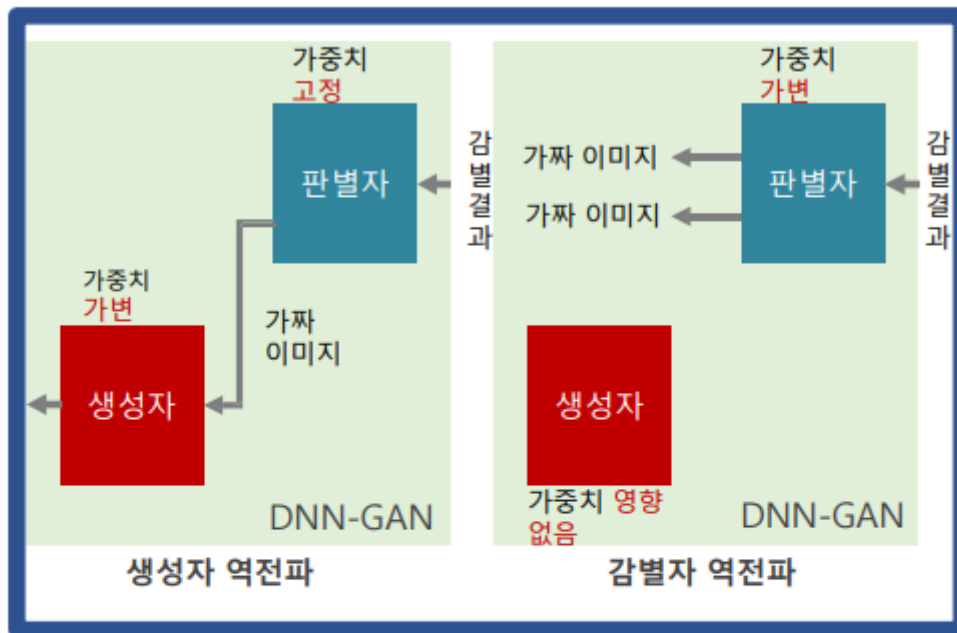
- 입력: 노이즈 벡터 (생성자의 입력)
- 출력: 감별 결과 (감별자의 출력)
- 생성자의 출력이 감별자의 입력이 됨



▪ DNN-GAN 설계도

- DNN-GAN은 생성자와 감별자를 연결하면서 완성됨
- DNN-GAN 입력: 노이즈 벡터 (생성자의 입력)
- DNN-GAN 출력: 감별 결과 (감별자의 출력)
- DNN-GAN 내부에서는 생성자의 출력이 감별자의 입력이 됨

2) DNN-GAN 역전파 방식



	생성자	감별자
필요 사항	감별자 필요	생성자 불필요
가중치 고정	감별자 고정 필요	생성자 고정 불필요
compile	DNN-GAN 전체	감별자만

1) train_on_batch 이론

(1) GAN에서 사용하는 학습법

- "Runs a single gradient update on a single batch of data"
- 한 batch로 학습하고, 한번 가중치 보정
- 입력: (1) 학습용 입력 데이터, (2) 학습용 출력 데이터
- 출력: 평균 손실값 (한 batch당)

	train_on_batch	fit
입력 데이터	batch 하나	학습용 데이터 전체
가중치 보정	매 batch마다 한번	매 batch마다 한번
주 목적	batch 당 추가 정보 계산	모든 데이터로 전체 학습 진행
epoch	무의미	중요한 파라미터

1. 감별자/생성자 학습 이론

- train_on_batch 이론
 - GAN에서 사용하는 학습법임
 - 감별자는 진짜 이미지로 한번 학습, 그리고 가짜 이미지로 학습, 총 2번 진행함
 - 감별자와 생성자 모두 train_on_batch 함수를 사용하여 학습 진행

2. 감별자/생성자 학습

- 감별자, 생성자 학습
 - Numpy의 zeros 함수는 감별자에게 주어진 이미지가 가짜임을 판별토록 유도
 - Numpy의 ones 함수는 감별자에게 주어진 이미지가 진짜임을 판별토록 유도

3. 샘플 이미지 생성

- 가짜 이미지 생성
 - 생성자가 얼마나 학습이 되었는지 판단을 위해 필요함
 - 가짜 작품을 만들기 위해 노이즈 벡터 사용함
- 이미지 파일 생성
 - Matplotlib의 subplots함수와 imshow 함수를 사용하여 가짜 이미지 출력함

4. DNN-GAN 학습

- GAN 학습 코딩
 - DNN-GAN에서는 매 batch마다 감별자 한번 학습, 그리고 생성자 한번 학습 진행함
 - 여러 epoch를 반복하며 감별자와 생성자를 번갈아 가며 학습함
- 컨트롤 타워
 - DNN-GAN 학습 진행 시, 종종 생성자에게 가짜 이미지를 생성하게 하여, 진행상황을 판단함